Отчёт: методы SASRec / StackRec, оптимизаторы и тестирование 06.11.2026

Tests_StackRec_AI4Science

06 ноября 2026

Amazon Beauty (reviews_Beauty_5), seed 42, batch_size=128

---

1. Метод SASRec (Self-Attentive Sequential Recommendations)

(Vaswani et al., 2017).

Архитектура

- **Embedding-слой:** item embeddings (padding_idx=0) + position embeddings.
- **Transformer Encoder:** `num_layers` слоёв (по умолчанию 6), каждый содержит:

- Multi-head self-attention (causal mask, `num_heads=1`).

- Position-wise feed-forward network (GELU, `dim_feedforward=64`).

- Dropout (`dropout=0.5`), LayerNorm (`eps=1e-9`).

- **Local objective:** для каждого элемента последовательности предсказывается следующий элемент через dot-product с таблицей item-эмбеддингов.
- **Inference:** top-k рекомендаций по последнему скрытому представлению.

Параметры модели (по умолчанию)

Параметр	Значение
---	---
embedding_dim	64
num_heads	1
num_layers	6

Параметр	Значение
dim_feedforward	64
dropout	0.5
activation	GELU

---

2. Метод StackRec (Stacked recommender System)

модели SASRec. Вместо обучения модели с нуля для каждого числа слоёв, StackRec обучает одну 6-слойную модель поэтапно, «распаковывая» её снизу вверх.

Три этапа

Этап	Активные слои	Новые слои	Копирование весов	Эпохи	LR
---	---	---	---	---	---
1 (2_layers)	4, 5	—	—	90	0.00297
2 (4_layers)	2–5	2, 3	4→2, 5→3	60 (+10 ramp)	0.00210
3 (6_layers)	0–5	0, 1	2→0, 3→1	60 (+10 ramp)	0.00171

Ключевые механизмы

каждый слой умножается на коэффициент. Значение 0 делает слой identity (пропускает вход), 1 — работает нормально. Это позволяет «включать» новые слои постепенно.

в начале каждого нового этапа веса уже обученных слоёв копируются в новые (например, `4→2` означает: веса слоя 4 копируются в слой 2). Это обеспечивает continuity — модель начинает новый этап с уже хорошей подсети.

в течение `ramp_epochs` (10) коэффициенты новых слоёв линейно растут от 0 до 1, обеспечивая плавный переход.

при добавлении новой группы слоёв добавляется вспомогательная потеря, которая заставляет выход новой подсети быть близким к входу (отношение → 1). Коэффициент

`lambda(alpha) = lambda0 * (1 - alpha)` линейно убывает вместе с ramp.

на границе каждой новой группы можно добавить голову с гиперболической геометрией (Poincaré ball или Lorentz hyperboloid) для учёта иерархии товаров.

Результаты (AdamW StackRec, test split)

Эффективные слои	val NDCG@10	test NDCG@10	val Recall@10	test Recall@10
---	---	---	---	---
2 (layers 4,5)	0.014714	0.011958	0.029630	0.023380
4 (layers 2-5)	0.015451	0.012964	0.031746	0.025370
6 (all layers)	0.016095	0.013299	0.033439	0.025494

StackRec значительно превосходит plain SASRec с фиксированным числом слоёв: для 4 слоёв test NDCG@10 = 0.012964 против 0.004797 (в 2.7 раза), для 6 слоёв — 0.013299 против 0.005645 (в 2.4 раза).

---

3. Оптимизаторы

3.1 AdamW

Стандартный оптимизатор с weight decay (decoupled weight decay).

Параметр	Значение
---	---
betas	(0.9, 0.999)
eps	1e-8
weight_decay	0.0
grad_clip	10.0

AdamW — базовый оптимизатор для всех экспериментов. Используется для embedding-параметров и 1D весов.

3.2 Muon

(Keller Jordan).

Алгоритм

1. Градиент 2D-матрицы `g` нормализуется.

2. Применяется 5-шаговая Newton-Schulz итерация для приближения `U @ V^T` (SVD `g = U @ S @ V^T`), т.е. все сингулярные значения push-ятся к 1.

3. Обновление масштабируется на `sqrt(max(1, rows/cols))` для сравнимости RMS с AdamW.

4. Применяется momentum (Nesterov, 0.95).

Параметр	Значение
---	---
momentum	0.95
nesterov	True
ns_steps	5
weight_decay	0.0

Mixed MuonAdamW

В проекте используется гибридный подход:

- **2D hidden weights** → Muon
- **Embeddings и 1D параметры** → AdamW

Результаты сравнения (test NDCG@10)

Слои	AdamW StackRec	Muon StackRec	Plain Muon
---	---	---	---
2	0.011958	0.004602	0.005187
4	0.012964	0.004518	0.003540
6	0.013299	0.004341	0.003692

AdamW значительно превосходит Muon на данном датасете. Muon + StackRec улучшает plain Muon, но остаётся ниже AdamW + StackRec.

---

4. Тестирование 6 ноября 2026

4.1 Консолидация результатов

объединить JSON-файлы результатов в один и перенести тесты в отдельную папку.

Выполнено:

- Создан `results/all_results.json` с 9 записями из `results/sources/`.
- Тесты перемещены в `tests/`.
- `sys.path` обновлён в скриптах тестов.

Результат:

- `json.load()` — парсится, 9 записей.
- `test_metrics.py`: **10/10 PASS**.
- `test_stackrec.py`: **14/17 PASS** (3 pre-existing failure: `FullPipelineShort`, `SASrecVsStackRec2L`, `SASrecVsStackRec2LLong`).

4.2 Fault-injection тесты метрик

добавить тесты с намеренно внесёнными ошибками для проверки обнаружения.

`tests/test_metrics_with_errors.py`

Внесённые ошибки:

1. `BuggyNDCGMetric`: discount `1/log2(rank+2)` вместо `1/log2(rank+1)`, плюс деление на 2.

2. Все метрики (NDCG, Recall, Coverage, Novelty, MeanRank, ILD) — поделены на 2.

3. `BuggyCoverageMetric`: только первые 10 пользователей + деление на 2.

.

Часть 1 — ошибки обнаруживаются (PASS):

Тест	Что проверяет
---	---
`BuggyNDCGFormula`	`buggy[0]=0.215338` ≠ `correct[0]=0.500000`
`AllMetricsHalved`	Все 6 метрик делятся на 2 (NDCG, Recall, Coverage, Novelty, MeanRank, ILD)

Тест	Что проверяет
`BuggyCoverageFirst10Users`	`buggy=0.250000` ≠ `correct=1.000000`, использовано 50 items (10 users × 5)
`CorrectExpectationsDetectMutations`	`ndcg[9]=0.315465` (correct 1.0), `coverage=0.25` (correct 1.0), `recall=0.4` (correct 0.8)

Часть 2 — ошибочные метрики ожидают корректные значения (FAIL):

Тест	Ожидал	Получил
---	---	---
`BuggyNDCGShouldFail`	1.0	0.315465
`HalvedNDCGShouldFail`	0.487914	0.243957
`HalvedRecallShouldFail`	0.800000	0.400000
`BuggyCoverageShouldFail`	1.000000	0.250000
`HalvedNoveltyShouldFail`	0.947977	0.473988
`HalvedMeanRankShouldFail`	3.500000	1.750000
`HalvedILDShouldFail`	0.472566	0.236283

4.3 Проверка метрик на малых батчах (1–2 пользователя)

проверить корректность всех метрик на батчах с 1 и 2 пользователями.

`tests/test_metrics_small_batch.py` (20 проверок)

Проверяемые метрики:

- NDCG, Recall, Coverage, MeanRank, Novelty, ILD
- Групповые метрики по популярности (`*_group0`, `*_group1`)
- `compute_popularity_groups`
- End-to-end `inference()` на 1–2 пользователях
- Mutation testing: halved metrics, wrong NDCG discount, first-user-only Coverage

`MeanRankMetric` выдавал `mean_rank`, но агрегатор искал `meanrank@k`. Исправлено в `src/utils.py`: name resolver теперь сравнивает имена без учёта подчёркиваний и обрабатывает `_groupN` суффиксы.

Результат:

- `test_metrics_small_batch.py`: **20 passed**.

- Все тесты вместе: **34 passed**.
- `test_metrics.py`: **10/10 PASS**.
- `test_metrics_with_errors.py`: **4/4 PASS**.

4.4 Проверка temporal split датасета

проверить, что train-события имеют более ранние таймстампы, чем test-события.

`tests/test_dataset_split.py`

Проверки:

- Train — точный префикс Test для каждого пользователя.
- `max(train.timestamp) < min(test-only.timestamp)` для всех общих пользователей.
- Аналогично для Train → Validation и Validation → Test.
- CSV-файлы отсортированы по времени.

.

- Train → Test: 6 797 общих пользователей, 0 нарушений.
- Train → Validation: 4 725 пользователей, 0 нарушений.
- Validation → Test: 2 604 пользователей, 0 нарушений.

4.5 Проверка копирования весов между этапами StackRec

убедиться, что StackRec копирует веса именно с предыдущего этапа.

`tests/test_stackrec_stage_copy.py`

Проверки:

1. Конфиг: этап 2 копирует `[4→2, 5→3]`, этап 3 — `[2→0, 3→1]`.

2. После этапа 2 слои 2,3 = слоям 4,5.

3. После обучения на этапе 2 и копирования на этап 3: слои 0,1 = обновлённым слоям 2,3 (не исходным 4,5).

. Копирование реализовано корректно.

4.6 Проверка роста метрик после второго этапа StackRec

убедиться, что метрики растут после добавления нового блока слоёв.

`tests/test_stackrec_results_growth.py`

этап 1 (2_layers) → этап 2 (4_layers) по метрикам:

- ndcg@10, recall@10, ndcg@100, recall@100 (val + test)

.

AdamW (основной результат):

Метрика	Val 2→4	Test 2→4
---	---	---
NDCG@10	0.014714 → 0.015920	0.011958 → 0.012934
Recall@10	0.029630 → 0.031958	0.023380 → 0.024251
NDCG@100	0.032305 → 0.035401	0.021223 → 0.022911
Recall@100	0.122751 → 0.135026	0.071633 → 0.075488

val NDCG@10 растёт (0.007215 → 0.007884), но test NDCG@10 слегка падает (0.004602 → 0.004518).

---

5. Итоговая сводка тестов 06.11.2026

Тест	Файл	Результат
---	---	---
Консолидация результатов	—	OK, 9 записей
Метрики (базовые)	`test_metrics.py`	10/10 PASS
Fault-injection	`test_metrics_with_errors.py`	4 PASS, 7 FAIL
Малые батчи (1–2 user)	`test_metrics_small_batch.py`	20 passed
Temporal split	`test_dataset_split.py`	4 passed
StackRec stage copy	`test_stackrec_stage_copy.py`	2 passed
Metric growth	`test_stackrec_results_growth.py`	2 passed
StackRec (полный)	`test_stackrec.py`	14/17 PASS (3 pre-existing)

Тест	Файл	Результат
Всего	**59 passed, 10 failed (7 fault-injection), 3 pre-existing**	

`meanrank@k` всегда был 0.0 из-за mismatch имени метрики (`mean_rank` vs `meanrank`). Исправлено в `src/utils.py`.